

UNIT III

PROCESS SYNCHRONISATION AND DEADLOCKS

OBJECTIVES:

The Course will enable learners to:

- ▶ • Explain the basic concepts of operating systems and process.
- ▶ • Discuss threads and analyse various CPU scheduling algorithms.
- ▶ • Describe the concept of process synchronization and deadlocks.
- ▶ • Analyse various memory management schemes.
- ▶ • Describe I/O management and file systems.

SYLLABUS

- Process Synchronization: The critical-section problem - Peterson's Solution, Synchronization hardware, Mutex locks, Semaphores, monitors - Classic problems of synchronization: Bounded Buffer Problem - Reader's & Writer Problem, Dining Philosopher Problem. Deadlock: System model - Deadlock characterization, Methods for handling deadlocks - Deadlock prevention - Deadlock avoidance - Deadlock detection - Recovery from deadlock

OUTCOMES:

Upon completion of the course, the students will be able to:

- ▶ CO1: Demonstrate the basic concepts of operating systems and process.
- ▶ CO2: Implement process management techniques using inter-process communication.
- ▶ **CO3: Implement the concepts of process synchronization and deadlocks.**
- ▶ CO4: Apply various memory management schemes for the suitable scenario.
- ▶ CO5: Describe various I/O and file management techniques.
- ▶ CO6: Develop practical skills in developing system-level programming.

Process Synchronization

- ❖ Process Synchronization is a mechanism in operating systems used to manage the execution of multiple processes that access shared resources.
- ❖ Its main purpose is to ensure data consistency, prevent race conditions and avoid deadlocks in a multi-process environment.
- ▶ On the basis of synchronization, processes are categorized as one of the following two types:
- ▶ **Independent Process:** The execution of one process does not affect the execution of other processes.
- ▶ **Cooperative Process:** A process that can affect or be affected by other processes executing in the system.
- ▶ Process Synchronization is the coordination of multiple cooperating processes in a system to ensure controlled access to shared resources, thereby preventing race conditions and other synchronization problems.

- ▶ Improper Synchronization in Inter Process Communication Environment leads to following problems:
- ▶ **Inconsistency:** When two or more processes access shared data at the same time without proper synchronization. This can lead to conflicting changes, where one process's update is overwritten by another, causing the data to become unreliable and incorrect.
- ▶ **Loss of Data:** Loss of data occurs when multiple processes try to write or modify the same shared resource without coordination. If one process overwrites the data before another process finishes, important information can be lost, leading to incomplete or corrupted data.
- ▶ **Deadlock:** Lack of proper Synchronization leads to Deadlock which means that two or more processes get stuck, each waiting for the other to release a resource. Because none of the processes can continue, the system becomes unresponsive and none of the processes can complete their tasks.

- ▶ **Role of Synchronization in IPC**
- ▶ **Preventing Race Conditions:** Ensures processes don't access shared data at the same time, avoiding inconsistent results.
- ▶ **Mutual Exclusion:** Allows only one process in the critical section at a time.
- ▶ **Process Coordination:** Lets processes wait for specific conditions (e.g., producer-consumer).
- ▶ **Deadlock Prevention:** Avoids circular waits and indefinite blocking by using proper resource handling.
- ▶ **Safe Communication:** Ensures data/messages between processes are sent, received and processed in order.
- ▶ **Fairness:** Prevents starvation by giving all processes fair access to resources.

► Types of Process Synchronization

► The two primary type of process Synchronization in an Operating System are:

► **Competitive:** Two or more processes are said to be in Competitive Synchronization if and only if they compete for the accessibility of a shared resource.

Lack of Proper Synchronization among Competing process may lead to either Inconsistency or Data loss.

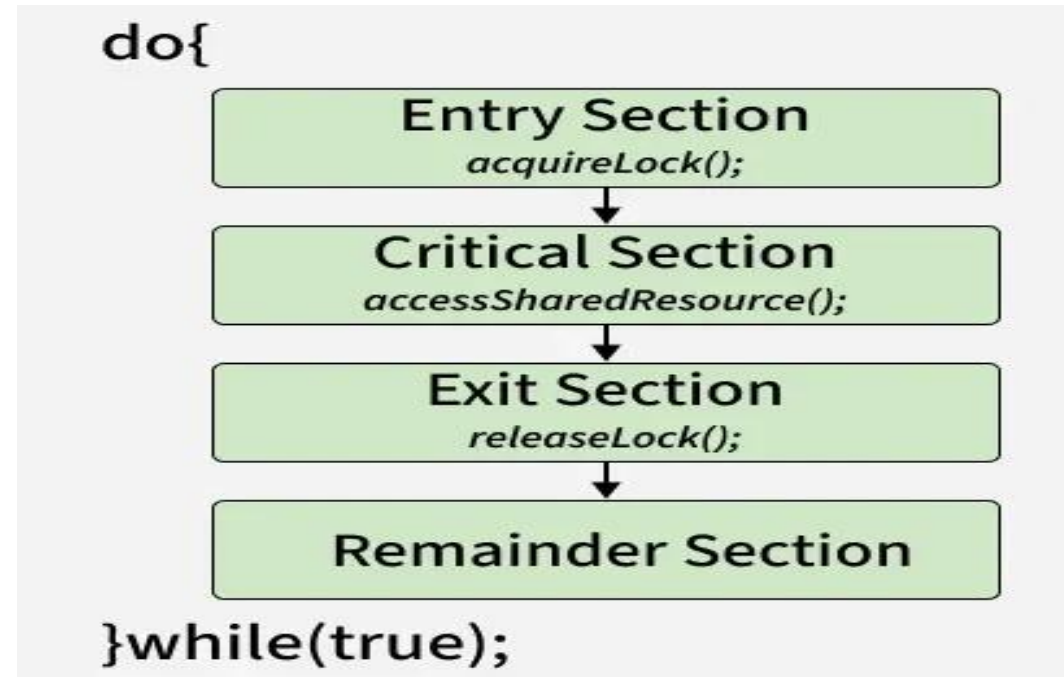
► **Cooperative:** Two or more processes are said to be in Cooperative Synchronization if and only if they get affected by each other i.e. execution of one process affects the other process.

Lack of Proper Synchronization among Cooperating process may lead to Deadlock.

The critical-section problem

- ▶ The Critical Section Problem in Operating Systems is managing concurrent access to shared resources (data, files, devices) by multiple processes to prevent data inconsistency (race conditions), ensuring only *one* process runs in its "critical section" (the code accessing shared data) at a time, requiring solutions like semaphores, mutexes, and monitors to guarantee mutual exclusion, progress, and bounded waiting.
- ▶ **Example**
Two processes updating the same bank account balance at the same time.

Structure of a Critical Section



► 1. Entry Section

► The process requests permission to enter the critical section.

► Synchronization tools (e.g., mutex, semaphore) are used to control access.

► **2. Critical Section:** The actual code where shared resources are accessed or modified.

► **3. Exit Section:** The process releases the lock or semaphore, allowing other processes to enter the critical section.

► **4. Remainder Section:** The rest of the program that does not involve shared resource access.

Critical Section Problem

► Shared Resources and Race Conditions

► Shared resources include memory, global variables, files, and databases.

► A **race condition** occurs when two or more processes attempt to update shared data at the same time, leading to unexpected results. **Example:** Two bank transactions modifying the same account balance simultaneously without synchronization may lead to incorrect final balance.

```
do{  
    flag=1;  
    while(flag); // (entry section)  
    // critical section  
    if (!flag)  
        // remainder section  
} while(true);
```

Requirements of a Solution

- ▶ A good critical section solution must ensure:
- ▶ **Correctness** - Shared data should remain consistent.
- ▶ **Efficiency** - Should minimize waiting and maximize CPU utilization.
- ▶ **Fairness** - No process should be unfairly delayed or starved.

Requirements of Critical Section Solutions

▶ 1. Mutual Exclusion

- ▶ At most one process can be inside the critical section at a time.
- ▶ Prevents conflicts by ensuring no two processes update the shared resource simultaneously.

▶ 2. Progress

- ▶ If no process is in the critical section, and some processes want to enter, the choice of who enters next should not be postponed indefinitely.
- ▶ Ensures that the system continues to make progress rather than getting stuck.

▶ 3. Bounded Waiting

- ▶ There must be a limit on how long a process waits before it gets a chance to enter the critical section.
- ▶ Prevents **starvation**, where one process is repeatedly bypassed while others get to execute.

- ▶ **Solution to Critical Section Problem :**

- ▶ A simple solution to the critical section can be thought of as shown below,

- ▶ *acquireLock();*
Process Critical Section
releaseLock();

- ▶ A thread must acquire a lock prior to executing a critical section. The lock can be acquired by only one thread.

Peterson's Algorithm in Process Synchronization

- ▶ Peterson's Solution is a software-based method used in operating systems to achieve **mutual exclusion** in a **critical section problem** for two processes.
- ▶ It ensures that only one process accesses the shared resource at a time, preventing race conditions.

The algorithm uses two shared variables:

- ▶ **flag[i]**: shows whether process i wants to enter the critical section.
- ▶ **turn**: indicates whose turn it is to enter if both processes want to access the critical section at the same time.

Similarly, Structure of Process Pj will be.

```
do
{
    // Critical Section
    // It means process pi is ready to enter its critical section
    flag[j] = true;

    turn = i; // It means that if pi wants to enter then allow it to enter and pj will wait

    // Condition to check if the flag of pi is true and turn is equal to pi, this will only break when
    one of the conditions gets false.
    while (flag[i] && turn == [i]);

    // Remainder Section
    //it sets the flag of pj to false because it has completed its critical section.
    flag[j] = false;
} while (TRUE);
```

► Explanation of Peterson's Algorithm

- Firstly, both processes set their flag variables to indicate that they don't currently want to enter the critical section. By default, the turn variable is set to the ID of one of the processes, it can be 0 or 1. This will indicate that it is initially the turn of that process to enter the critical section.
- Both processes set their flag variable to indicate their intention to enter the critical section.
- Then the process, which is having the next turn, sets the turn variable to its own ID. This will indicate that it is its turn to enter the critical section. For example, if the current value of turn is 1, and it is now the turn of Process 0 to enter, Process 0 sets $\text{turn} = 0$.
- Both processes enter a loop where they will check the flag of the other process and wait if necessary. The loop continues as long as the following two conditions are met:
 - i. The other process has expressed its intention to enter the critical section ($\text{flag}[1 - \text{processID}] == \text{true}$

for Process processID).

ii. It is currently the turn of the other process ($\text{turn} == 1 - \text{processID}$).

If both conditions are satisfied, the process waits and yields the CPU to the other process. This ensures that the other process has an opportunity to enter the critical section.

- Once a process successfully exits the waiting loop, then it can enter the critical section. It can also access the shared resource without interference from the other process. It can perform any necessary operations or modifications within this section.
- After completing its work in the critical section, the process resets its flag variable. Resetting is required to indicate that this process no longer wants to enter the critical section ($\text{flag}[\text{processID}] = \text{false}$). This step ensures that the process can enter the waiting loop again correctly if needed.

Synchronization hardware

- ▶ Process Synchronization problems occur when two processes running concurrently share the same data or same variable.
- ▶ The value of that variable may not be updated correctly before its being used by a second process. Such a condition is known as Race Around Condition.
- ▶ There are a software as well as hardware solutions to this problem.

There are three algorithms in the hardware approach of solving Process Synchronization problem:

- ▶ Test and Set
- ▶ Swap
- ▶ Unlock and Lock

1. Test and Set:

- ▶ Here, the shared variable is lock which is initialized to false.
- ▶ TestAndSet(lock) algorithm works in this way - it always returns whatever value is sent to it and sets lock to true.
- ▶ The first process will enter the critical section at once as TestAndSet(lock) will return false and it'll break out of the while loop.
- ▶ The other processes cannot enter now as lock is set to true and so the while loop continues to be true. Mutual exclusion is ensured.
- ▶ Once the first process gets out of the critical section, lock is changed to false. So, now the other processes can enter one by one. Progress is also ensured.
- ▶ However, after the first process, any process can go in. There is no queue maintained, so any new process that finds the lock to be false again can enter.
- ▶ So bounded waiting is not ensured.

Test and Set Pseudocode -

```
//Shared variable lock initialized to false
boolean lock;

boolean TestAndSet (boolean &target){
    boolean rv = target;
    target = true;
    return rv;
}

while(1){
    while (TestAndSet(lock));
    critical section
    lock = false;
    remainder section
}
```

2. Swap:

- ▶ *Swap function* uses two boolean variables lock and key.
- ▶ Both lock and key variables are initially initialized to false.
- ▶ Swap algorithm is the same as lock and set algorithm.
- ▶ The Swap algorithm uses a temporary variable to set the lock to true when a process enters the critical section of the program.

boolean lock;
Individual key;

```
void swap(boolean &a, boolean &b){  
    boolean temp = a;  
    a = b;  
    b = temp;  
}
```

```
while (1){  
    key = true;
```

```
    while(key)  
        swap(lock,key);  
    critical section  
    lock = false;  
    remainder section  
}
```

- ▶ ***Unlock and lock algorithm*** uses the TestAndSet method to control the value of lock.
- ▶ Unlock and lock algorithm uses a variable waiting[i] for each process i.
- ▶ Here i is a positive integer i.e 1,2,3,... which corresponds to processes P1, P2, P3... and so on.
- ▶ waiting[i] checks if the process i is waiting or not to enter into the critical section.
- ▶ All the processes are maintained in a ready queue before entering into the critical section.
- ▶ The processes are added to the queue with respect to their process number.
- ▶ The queue is the circular queue.

Mutex Locks

- ▶ A **Mutex Lock** (Mutual Exclusion Lock) is a **synchronization mechanism** used in an operating system to **prevent multiple processes or threads from accessing a shared resource at the same time**.

Why Mutex Locks are Needed

- ▶ In a **multithreaded or multiprocess** environment, concurrent access to shared data can cause **race conditions**, leading to incorrect results.
- ▶ Mutex locks ensure **only one thread enters the critical section at a time**.
- ▶ A mutex has **two states**:
- ▶ **Locked (0)** - resource is busy
- ▶ **Unlocked (1)** - resource is free
- ▶ Only the thread that **locks** the mutex can **unlock** it.

Working of Mutex Lock

- ▶ Thread requests the mutex using `lock()`
- ▶ If mutex is **free**, the thread enters the critical section
- ▶ If mutex is **busy**, the thread is blocked (or spins)
- ▶ After finishing, the thread calls `unlock()`
- ▶ Another waiting thread gets the mutex

Semaphores in Process Synchronization

- ▶ In multiprogramming systems, multiple processes may need to access shared resources like files, printers, or memory. To handle this, operating systems use synchronization mechanisms. One of the most widely used mechanisms is the Semaphore.
- ▶ A Semaphore is simply a variable (integer) used to control access to a shared resource by multiple processes in a concurrent system. It ensures that only the allowed number of processes can use a resource at a given time.
- ▶ A semaphore works using two fundamental operations:
 - ▶ **1. Wait** (P operation / down)
 - ▶ Decreases the semaphore value.
 - ▶ If the value becomes negative, the process is blocked until the resource becomes available.
 - ▶ **2. Signal** (V operation / up)
 - ▶ Increases the semaphore value.
 - ▶ If there are waiting processes, one of them gets unblocked.

```
wait(Semaphore S){  
    while (S == 0);  
    S = S - 1;           # acquire resource  
}  
  
[ Critical Section ]  
  
signal(Semaphore S){  
    S = S + 1;           # release resource  
}
```

► Features of Semaphores

- **Mutual Exclusion:** Semaphore ensures that only one process accesses a shared resource at a time.
- **Process Synchronization:** Semaphore coordinates the execution order of multiple processes.
- **Resource Management:** Limits access to a finite set of resources, like printers, devices, etc.
- **Reader-Writer Problem:** Allows multiple readers but restricts the writers until no reader is present.
- **Avoiding Deadlocks:** Prevents deadlocks by controlling the order of allocation of resources.

► **Types of Semaphores**

► Semaphores are mainly of two Types:

► **1. Counting Semaphore**

► Used when multiple instances of a resource exist.

► The semaphore value can range over an unrestricted domain (0 to N).

► Example: Managing access to a pool of 5 printers.

► **2. Binary Semaphore**

► Special case of counting semaphore with only two values: 0 and 1.

► Works like a lock: either the resource is free (1) or busy (0).

► Example: Managing access to a single critical section.

Classic problems of synchronization

- ▶ Bounded Buffer Problem
- ▶ Reader's & Writer Problem
- ▶ Dining Philosopher Problem
- ▶ **The Producer-Consumer problem** is a classic example of a synchronization problem in operating systems. It demonstrates how processes or threads can safely share resources without conflicts. This problem belongs to the process synchronization domain, specifically dealing with coordination between multiple processes sharing a common buffer.
- ▶ In this problem, we have:
- ▶ **Producers:** Generate data items and place them in a shared buffer.
- ▶ **Consumers:** Remove and process data items from the buffer.
- ▶ The main challenge is to ensure:
- ▶ A producer does not add data to a full buffer.
- ▶ A consumer does not remove data from an empty buffer.
- ▶ Multiple producers and consumers do not access the buffer simultaneously, preventing race conditions.

- ▶ **Problem Statement**
- ▶ **Producer:** produces items and puts them into a **shared buffer**
- ▶ **Consumer:** removes items from the same buffer
- ▶ Buffer has **limited size**
- ▶ Producer must **not add** if buffer is full
- ▶ Consumer must **not remove** if buffer is empty
- ▶ Normally, semaphores are used — but here we solve it **without semaphores**, using **shared variables (busy waiting)**.

```
#define BUFFER_SIZE 5
```

```
int buffer[BUFFER_SIZE];
```

```
int in = 0, out = 0, count = 0;
```

Producer Code (Without Semaphore)

```
while (true)
```

```
{
```

```
    // Produce an item
```

```
    item = produce_item();
```

```
    // Busy waiting if buffer is full
```

```
    while (count == N)
```

```
        ; // do nothing
```

```
    buffer[in] = item;
```

```
    in = (in + 1) % N;
```

```
    count++;
```

```
}
```

1. while (true)

This creates an infinite loop

The producer continuously produces items

2. item = produce_item();

The producer creates a new item

This item can be data, message, job, etc.

Happens outside the critical section

3. while (count == N) ;

Busy waiting condition

count → number of items currently in the buffer

N → maximum buffer size

✦ **Meaning:**

If count == N, buffer is full

Producer keeps checking the condition repeatedly

CPU is wasted while waiting

4. buffer[in] = item;

Producer places the item into the buffer

in indicates the next free position

This is part of the critical section

5. in = (in + 1) % N;

Updates buffer index in a circular manner

Ensures index stays within buffer size

6. count++;

Increments the number of items in the buffer

Helps consumer know buffer is not empty

Consumer(without semaphores)

while (true)

{

// Busy waiting if buffer is empty

while (count == 0)

; // do nothing

item = buffer[out];

out = (out + 1) % N;

count--;

consume_item(item);

}

1. while (true)

Creates an infinite loop

The consumer continuously tries to consume items

2. while (count == 0) ;

Busy waiting loop

count indicates number of items in the buffer

Meaning:

If count == 0, buffer is **empty**

Consumer keeps checking the condition repeatedly

CPU time is **wasted** during waiting

3. item = buffer[out];

Consumer removes an item from the buffer

out indicates the **position of the next item to consume**

This statement is part of the **critical section**

4. out = (out + 1) % N;

Updates the buffer index in a **circular fashion**

Prevents array index overflow

Example:

If N = 5 and out = 4

$(4 + 1) \% 5 = 0 \rightarrow$ wraps around

5. count--;

Decreases the number of items in the buffer

Signals that one buffer slot has become **free**

6. consume_item(item);

The consumer uses the consumed **item**

Happens **outside the critical section**

Reader's & Writer Problem

- ▶ The Readers-Writers Problem is a classic synchronization issue in operating systems.
- ▶ It deals with coordinating access to shared data (e.g., database, file) by multiple processes or threads.
- ▶ **Readers:** Multiple readers can read the shared data **simultaneously** without causing inconsistency (since they don't modify data).
- ▶ **Writers:** Only **one writer** can access the data at a time, and no readers are allowed while writing (to prevent data corruption).
- ▶ The challenge is to design a synchronization scheme that ensures:
- ▶ Multiple readers can access data together if no writer is writing.
- ▶ Writers have exclusive access no other reader or writer can enter during writing.
- ▶ **Variants of the Problem**
- ▶ **1. Readers Preference**
- ▶ Readers are given priority.
- ▶ No reader waits if the resource is available for reading, even if a writer is waiting.
- ▶ Writers may suffer from *starvation*.
- ▶ **2. Writers Preference**
- ▶ Writers are prioritized over readers.
- ▶ Ensures writers won't starve, but readers may wait longer.

Rules of the Problem

- ▶ Multiple readers can read the shared resource **simultaneously**.
- ▶ **Only one writer** can access the shared resource at a time.
- ▶ **No reader** can access the resource **while a writer is writing**.
- ▶ Writers require **exclusive access**.

Solution When Reader Has The Priority Over Writer

- ▶ Here priority means, no reader should wait if the shared resource is currently open for reading. There are four types of cases that could happen here.

Case	Process 1	Process 2	Allowed/Not Allowed
Case 1	Writing	Writing	Not Allowed
Case 2	Writing	Reading	Not Allowed
Case 3	Reading	Writing	Not Allowed
Case 4	Reading	Reading	Allowed

Synchronization Variables

- ▶ **mutex**: Ensures mutual exclusion when updating the reader count (readcnt).
- ▶ **wrt**: Controls access to the shared data (used by both readers and writers).
- ▶ **readcnt**: Number of readers currently in the critical section.

Semaphore Operations

- ▶ **wait()**: Decreases semaphore value, blocks if < 0 .
- ▶ **signal()**: Increases semaphore value, wakes waiting process if any.

Reader Process(Reader Preference)

- ▶ 1. Reader requests the entry to critical section.
- ▶ 2. If allowed:
 - ▶ it increments the count of number of readers inside the critical section. If this reader is the first reader entering, it locks the **wrt** semaphore to restrict the entry of writers if any reader is inside.
 - ▶ It then, signals **mutex** as any other reader is allowed to enter while others are already reading.
 - ▶ After performing reading, it exits the critical section. When exiting, it checks if no more reader is inside, it signals the semaphore "wrt" as now, writer can enter the critical section.
- ▶ 3. If not allowed, it keeps on waiting.

Reader-Writer Problem Using Semaphores

Shared Variables

```
int readcount = 0;  
semaphore mutex = 1; // Protects readcount  
semaphore wrt = 1; // Controls access to shared  
resource
```

Reader Process

```
while (true)  
{  
    wait(mutex);  
    readcount++;  
    if (readcount == 1)  
        wait(wrt); // First reader blocks writers  
    signal(mutex);  
    // Reading the shared data  
    read_data();  
    wait(mutex);
```

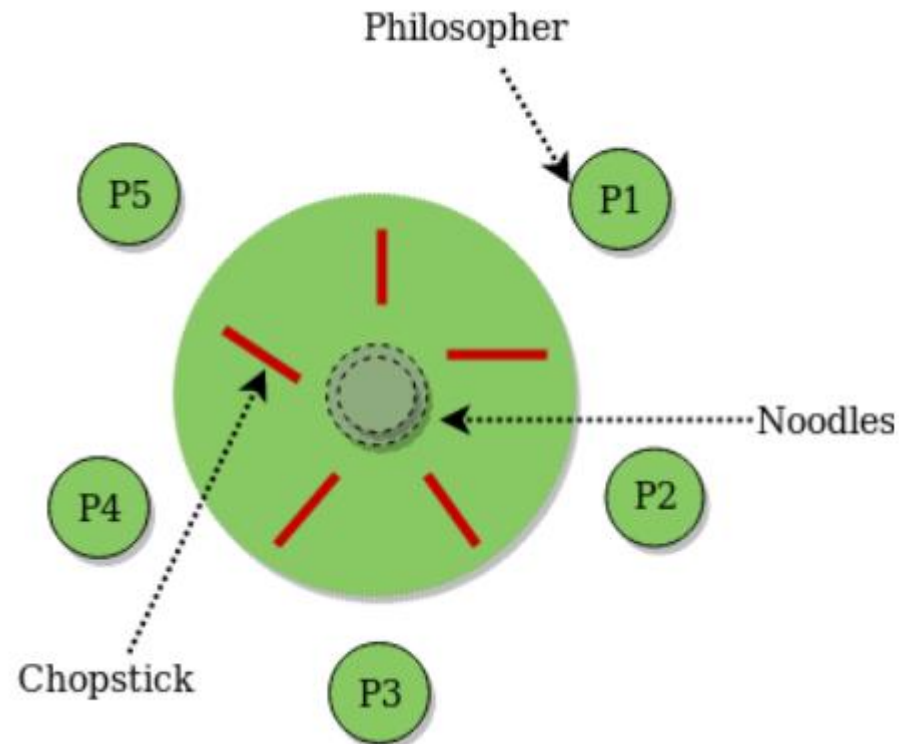
```
    readcount--;  
    if (readcount == 0)  
        signal(wrt); // Last reader allows writers  
    signal(mutex);  
}
```

Writer Process

```
▶ while (true)
▶ {
▶     wait(wrt);      // Writer gets exclusive access
▶     // Writing to shared data
▶     write_data();
▶     signal(wrt);
▶ }
```

Dinning Philosopher Problem

- ▶ The **Dining Philosopher Problem** is a classic synchronization and concurrency problem that deals with resource sharing, deadlock, and starvation in systems where multiple processes require limited resources.
- ▶ The **Dining Philosopher Problem** involves 'n' philosophers sitting around a circular table. Each philosopher alternates between two states: **thinking** and **eating**. To eat, a philosopher needs two chopsticks, one on their left and one on their right. However, the number of chopsticks is equal to the number of philosophers, and each chopstick is shared between two neighboring philosophers.
- ▶ The standard problem considers the value of 'n' as 5 i.e. we deal with 5 Philosophers sitting around a circular table.



- ▶ **The challenge:** Design a synchronization mechanism so that philosophers can eat without causing **deadlock** (all waiting forever) or **starvation** (some never get a chance to eat).

Issues in the Problem

- ▶ **Deadlock:** If every philosopher picks up their left chopstick first, no one can pick up the right one circular wait.
- ▶ **Starvation:** Some philosophers may never get a chance to eat if others keep eating.
- ▶ **Concurrency Control:** Must ensure no two adjacent philosophers eat simultaneously.

Constraints and Conditions for the Problem

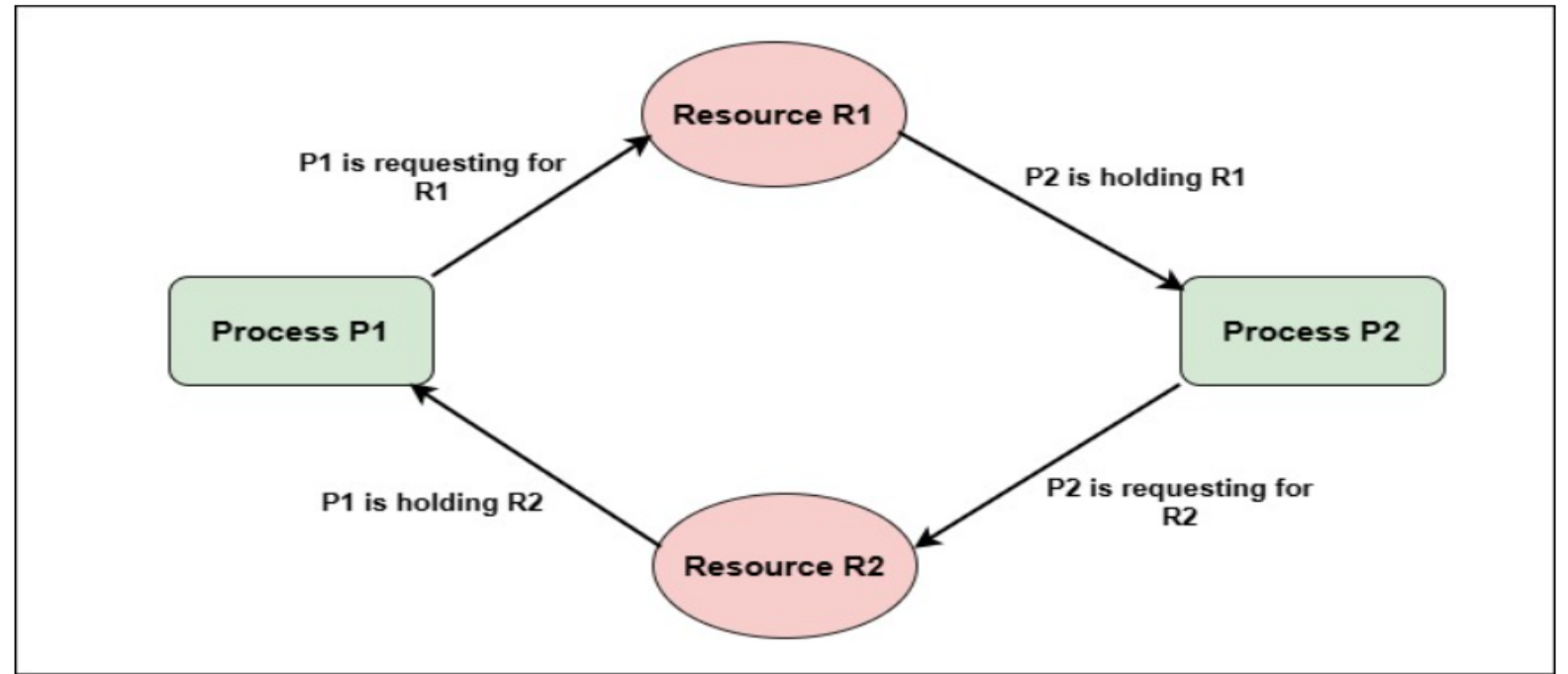
- ▶ Every Philosopher needs two forks to eat.
- ▶ Every Philosopher may pick up the forks on the left or right but only one fork at once.
- ▶ Philosophers only eat when they have two forks. We have to design such a protocol i.e. pre and post protocol which ensures that a philosopher only eats if he or she has two forks.
- ▶ Each fork is either clean or dirty.

Solution

- ▶ **Mutual Exclusion Principle:** No two Philosophers can have the two forks simultaneously.
- ▶ **Free from Deadlock:** Each philosopher can get the chance to eat in a certain finite time.
- ▶ **Free from Starvation:** When few Philosophers are waiting then one gets a chance to eat in a while.
- ▶ **No strict Alternation**
- ▶ **Proper utilization of time**

Deadlock

- ▶ Deadlock is a state in an operating system where two or more processes are stuck forever because each is waiting for a resource held by another.
- ▶ It happens only when four conditions exist:
 - ▶ mutual exclusion,
 - ▶ hold and wait,
 - ▶ no preemption,
 - ▶ circular wait.

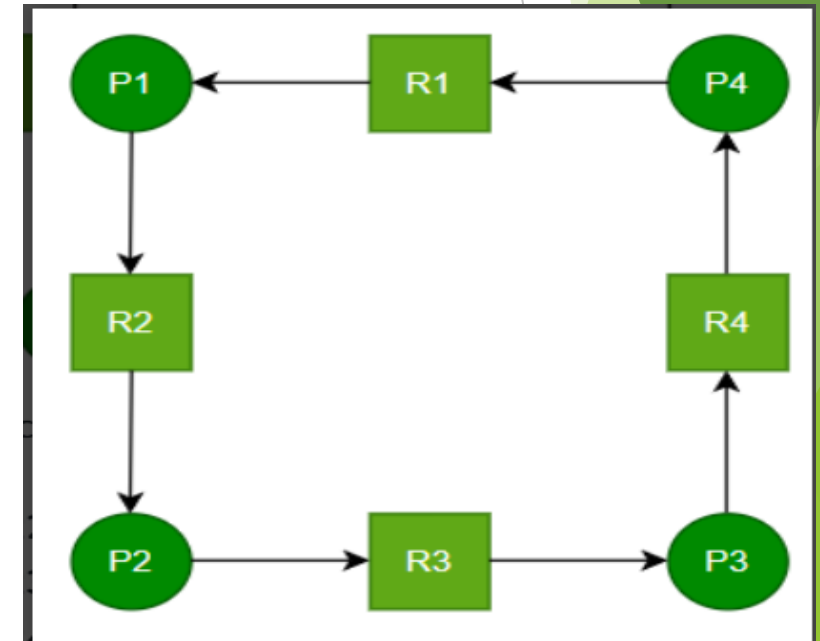


Necessary Conditions for Deadlock in OS

- ▶ Deadlock can arise if the following four conditions hold simultaneously (Necessary Conditions)
- ▶ **Mutual Exclusion:** Only one process can use a resource at any given time i.e. the resources are non-sharable.
- ▶ **Hold and Wait:** A process is holding at least one resource at a time and is waiting to acquire other resources held by some other process.
- ▶ **No Preemption:** A resource cannot be taken from a process unless the process releases the resource.
- ▶ **Circular Wait:** set of processes are waiting for each other in a circular fashion. For example, imagine four processes P1, P2, P3, and P4 and four resources R1, R2, R3, and R4.

The above image demonstrates a circular wait deadlock, here's how:

- ▶ P1 is holding R1 and waiting for R2 (which is held by P2).
- ▶ P2 is holding R2 and waiting for R3 (which is held by P3).
- ▶ P3 is holding R3 and waiting for R4 (which is held by P4).
- ▶ P4 is holding R4 and waiting for R1 (which is held by P1).

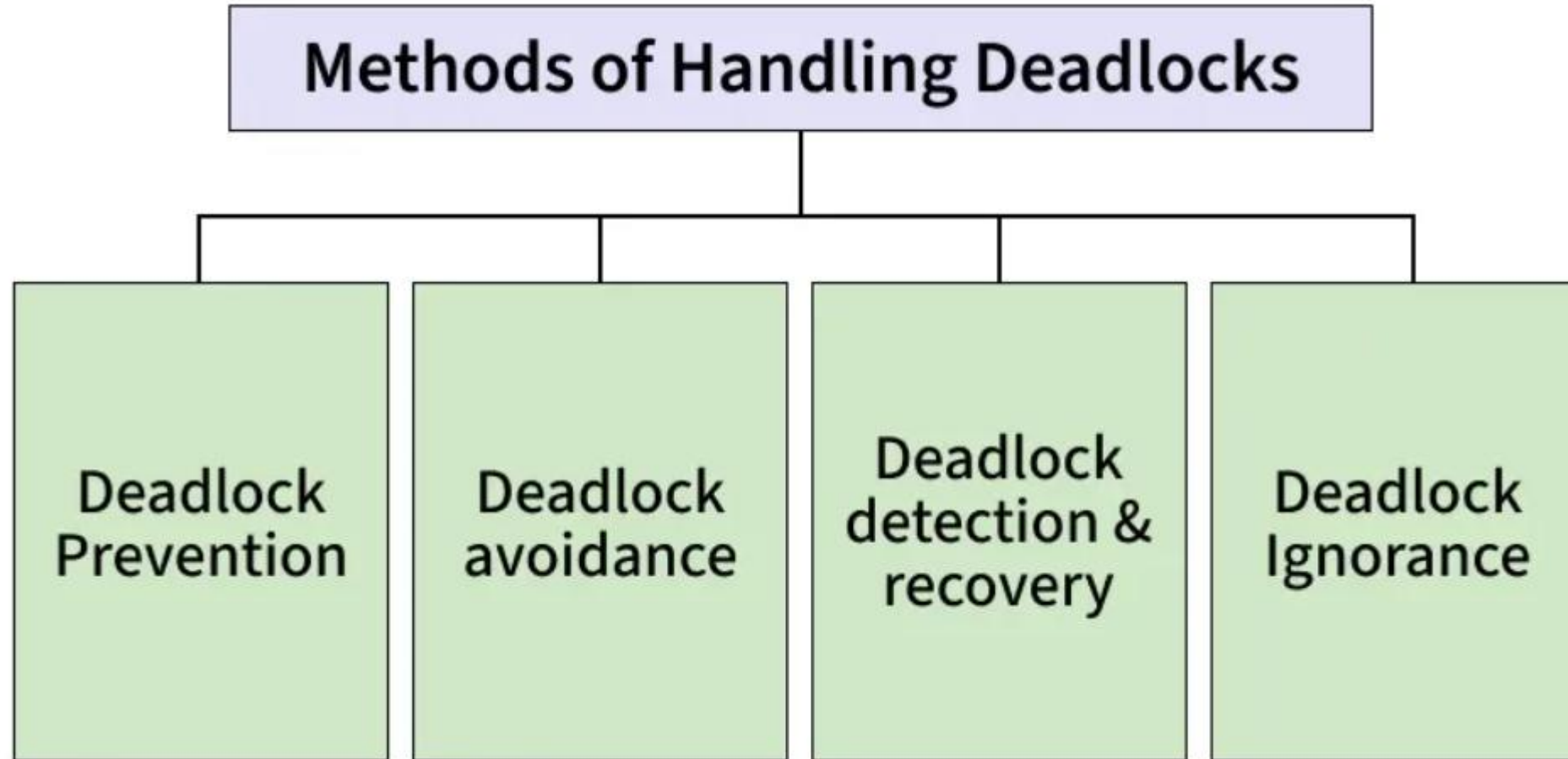


Methods for Handling Deadlocks :

In general, there are three approaches to dealing with deadlocks as follows.

- ▶ Preventing or avoiding deadlock by Avoid allowing the system to become stuck in a loop.
- ▶ Detection and recovery of deadlocks, When deadlocks are detected, abort the process or preempt some resources.
- ▶ Ignore the problem entirely.
- ▶ To avoid deadlocks, the system requires more information about all processes. The system, in particular, must understand what resources a process will or may request in the future. (Depending on the algorithm, this can range from a simple worst-case maximum to a complete resource request and release plan for each process.)
- ▶ Deadlock detection is relatively simple, but deadlock recovery necessitates either aborting processes or preempting resources, neither of which is an appealing option.
- ▶ If deadlocks are not avoided or detected, the system will gradually slow down as more processes become stuck waiting for resources that the deadlock has blocked and other waiting processes. Unfortunately, when the computing requirements of a real-time process are high, this slowdown can be confused with a general system slowdown.

Methods of Handling Deadlocks



Deadlock characterization

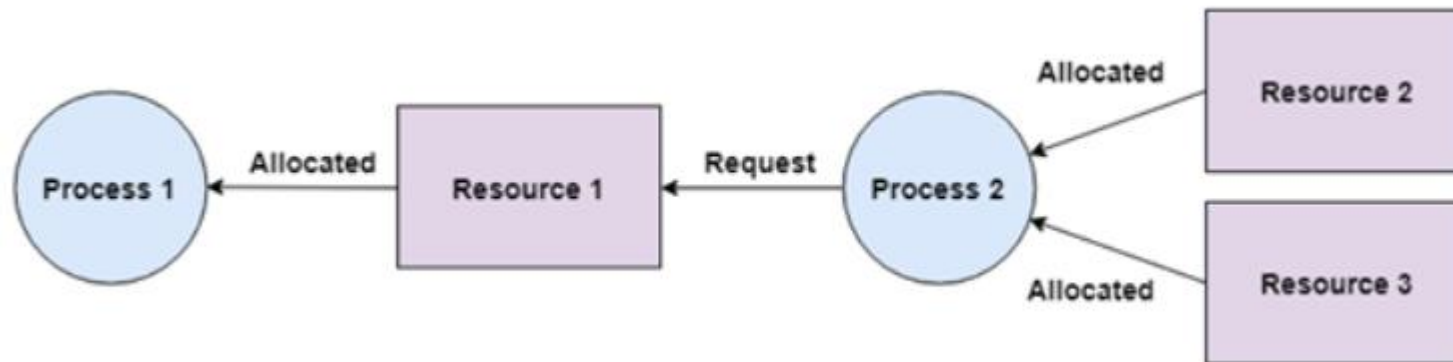
Mutual Exclusion

- There should be a resource that can only be held by one process at a time. In the diagram below, there is a single instance of Resource 1 and it is held by Process 1 only.



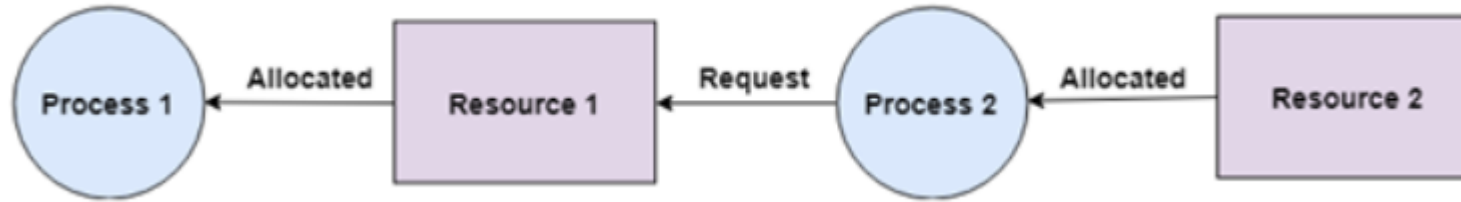
Hold and Wait

- A process can hold multiple resources and still request more resources from other processes which are holding them. In the diagram given below, Process 2 holds Resource 2 and Resource 3 and is requesting the Resource 1 which is held by Process 1.



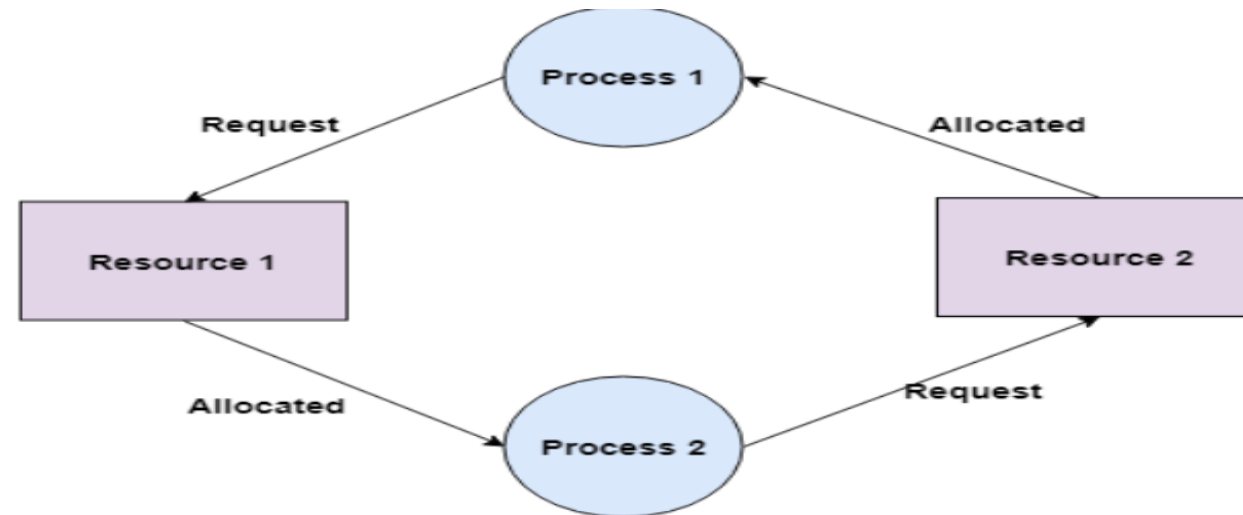
No Preemption

- ▶ A resource cannot be preempted from a process by force. A process can only release a resource voluntarily. In the diagram below, Process 2 cannot preempt Resource 1 from Process 1. It will only be released when Process 1 relinquishes it voluntarily after its execution is complete.



Circular Wait

- ▶ A process is waiting for the resource held by the second process, which is waiting for the resource held by the third process and so on, till the last process is waiting for a resource held by the first process. This forms a circular chain. For example: Process 1 is allocated Resource 2 and it is requesting Resource 1. Similarly, Process 2 is allocated Resource 1 and it is requesting Resource 2. This forms a circular wait loop.



Deadlock prevention

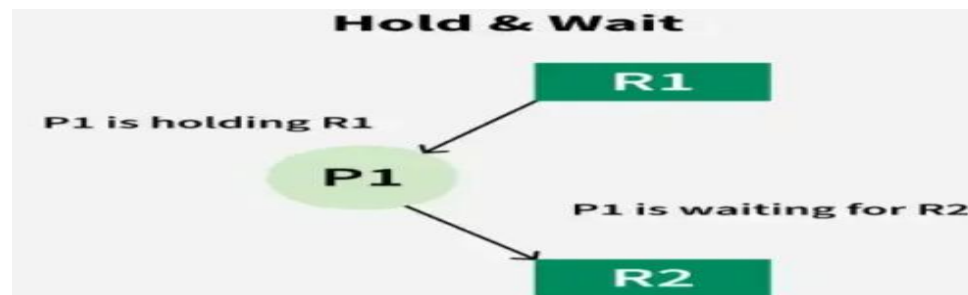
1. Eliminate Mutual Exclusion

- ▶ Only one process can use that resource at any given time. If another process requests the same resource, it must wait for the resource to be released.

2. Eliminate Hold and Wait

Hold and wait is a condition in which a process holds one resource while simultaneously waiting for another resource that is being held by a different process. The process cannot continue until it gets all the required resources.

- ▶ There are two ways to eliminate hold and wait:
- ▶ **By eliminating wait:** The process specifies the resources it requires in advance so that it does not have to wait for allocation after execution starts.
For Example, Process1 declares in advance that it requires both Resource1 and Resource2.
- ▶ **By eliminating hold:** The process has to release all resources it is currently holding before making a new request.
For Example: Process1 must release Resource2 and Resource3 before requesting Resource1.



3. Eliminate No Preemption

- ▶ No preemption means resources can't be taken away once allocated. To prevent this:
- ▶ **Processes must release resources voluntarily:** A process gives up resources once it finishes using them.
- ▶ **Avoid partial allocation:** If a process requests resources that are unavailable, it must release all currently held resources and wait until all required resources are free.

4. Eliminate Circular Wait

- ▶ Circular wait happens when processes form a cycle, each waiting for a resource held by the next. To prevent this:
- ▶ Impose a strict ordering on resources.
- ▶ Assign each resource a unique number.
- ▶ Processes can only request resources in increasing order of their numbers.
- ▶ This prevents cycles, as no process can go backwards in numbering.

Deadlock Avoidance

- ▶ Deadlock Avoidance is a method where the operating system makes decisions dynamically to ensure the system never enters an unsafe state, thereby avoiding the possibility of deadlock. This is usually done using algorithms like the Banker's Algorithm.
- ▶ Deadlock avoidance mainly relies on algorithms that check resource allocation before making decisions. The two common types are:
- ▶ **Banker's Algorithm:** Used when multiple instances of resources exist. It simulates allocation and ensures that the system remains in a safe state before granting resources.
- ▶ **Resource Allocation Graph (RAG) Algorithm:** Used when each resource has only one instance. It checks for cycles in the graph to avoid unsafe states.

Banker's Algorithm

- ▶ Banker's Algorithm is a resource allocation and deadlock avoidance algorithm used in operating systems. It ensures that a system remains in a safe state by carefully allocating resources to processes while avoiding unsafe states that could lead to deadlocks.
- ▶ The Banker's Algorithm is a smart way for computer systems to manage how programs use resources, like memory or CPU time.
- ▶ It helps prevent situations where programs get stuck and can not finish their tasks. This condition is known as deadlock.
- ▶ By keeping track of what resources each program needs and what's available, the banker algorithm makes sure that programs only get what they need in a safe order.

Components of the Banker's Algorithm

Available

A 1-D array showing how many instances of each resource type are currently free.

Max

A 2-D array showing the maximum number of each resource type each process may request.

Click to enlarge

Allocation

A 2-D array showing the number of each resource type currently allocated to each process.

Need

A 2-D array showing the remaining resources each process still requires to complete ($\text{Need} = \text{Max} - \text{Allocation}$).

- ▶ **Types of Banker's Algorithm**
- ▶ The Banker's Algorithm is made up of two parts:
- ▶ Safety Algorithm
- ▶ Resource Request Algorithm

Types of Banker's Algorithm

Safety Algorithm

Checks whether the system is in a safe state by determining if all processes can finish with the available resources.

Click to enlarge

Resource Request Algorithm

Determines whether a process's resource request can be safely granted without putting the system into an unsafe state

► Steps:

- 1. Initialize:
 - $Work = Available$ (resources currently available)
 - $Finish[i] = false$ for all processes (none have finished yet)
- 2. Look for a process P_i such that:
 - $Finish[i] = false$
 - $Need[i] \leq Work$ (resources required can be satisfied)
- 3. If such a process is found:
 - Pretend to allocate resources to it: $Work = Work + Allocation[i]$
 - Mark the process finished: $Finish[i] = true$
 - Repeat step 2 for remaining processes
- 4. If all processes are marked finished ($Finish[i] = true$ for all), the system is **safe**.

Example of the Banker's Algorithm

Example: Considering a system with five processes P_0 through P_4 and three resources of type A, B, C. Resource type A has 10 instances, B has 5 instances and type C has 7 instances. Suppose at time t_0 following snapshot of the system has been taken:

Process	Allocation	Max	Available
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Step 1:

Resource=Allocation + Available

Step 2:

Need= Max- Allocation

Step 3:

if need <= Available

(Work)

Available= Available +Allocation

else

Ignore it

- **Step 1: Calculate the Need Matrix**
The Need matrix is calculated as $\text{Need} = \text{Max} - \text{Allocation}$.

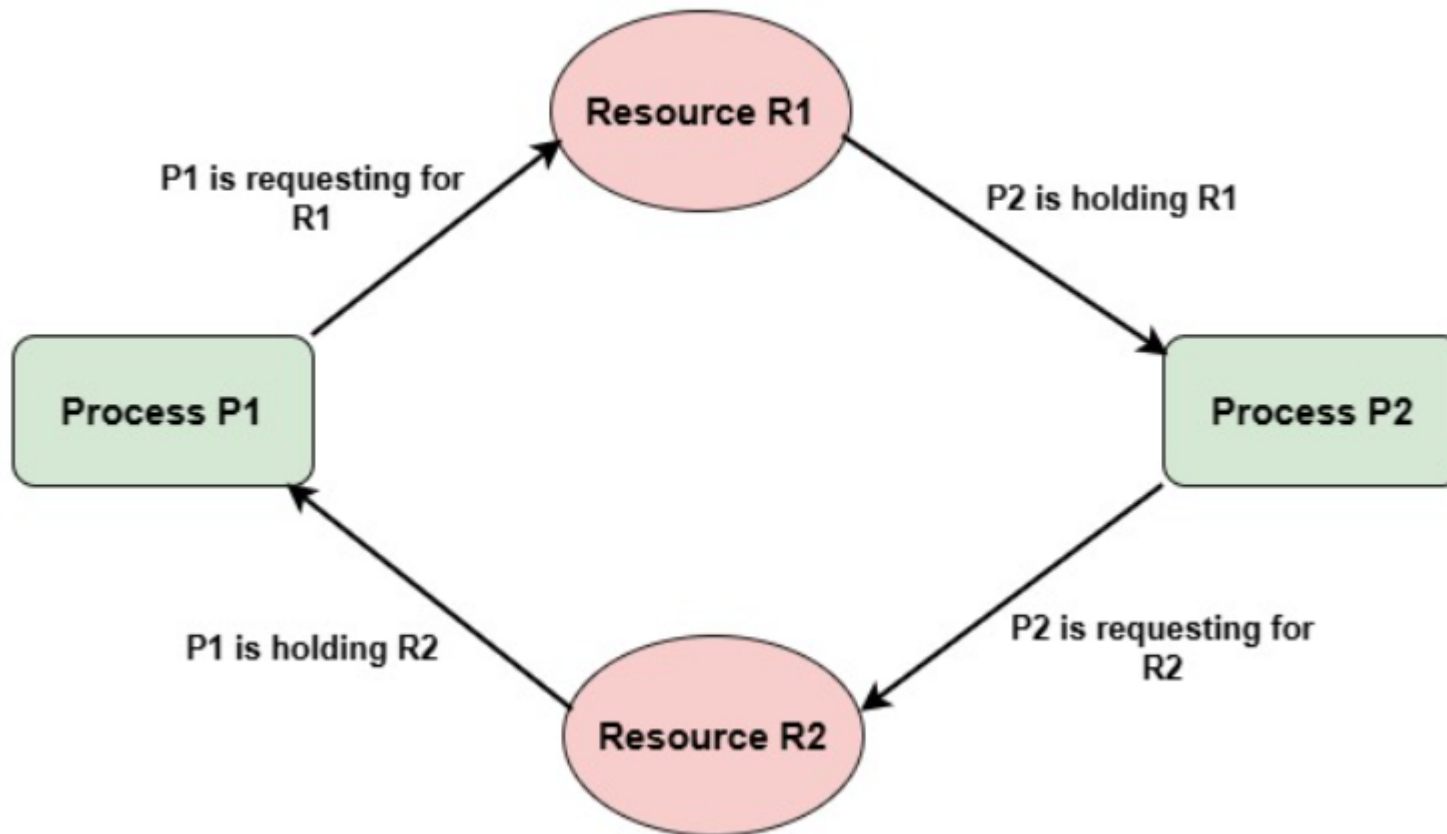
Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	1	2	2
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

Deadlock Detection and Recovery

- ▶ Deadlock detection and recovery is the mechanism of detecting and resolving deadlocks in an operating system.
- ▶ In operating systems, deadlock recovery is important to keep everything running smoothly.
- ▶ A deadlock occurs when two or more processes are blocked, waiting for each other to release the resources they need.
- ▶ Deadlock detection is the process of identifying when processes are stuck waiting for resources held by other processes.
- ▶ Recovery is the method of resolving the deadlock to allow the system to continue functioning.
- ▶ Detection is done using techniques like Resource Allocation Graphs (RAG) or Wait-for Graphs.
- ▶ Once a deadlock is detected, recovery methods include process termination, resource preemption, or process rollback.
- ▶ There are several algorithms used for deadlock detection, including –
 - ▶ **Resource Allocation Graph (RAG) Algorithm**
 - ▶ **Wait-for Graph Algorithm**
 - ▶ **Banker's Algorithm for Deadlock Detection**

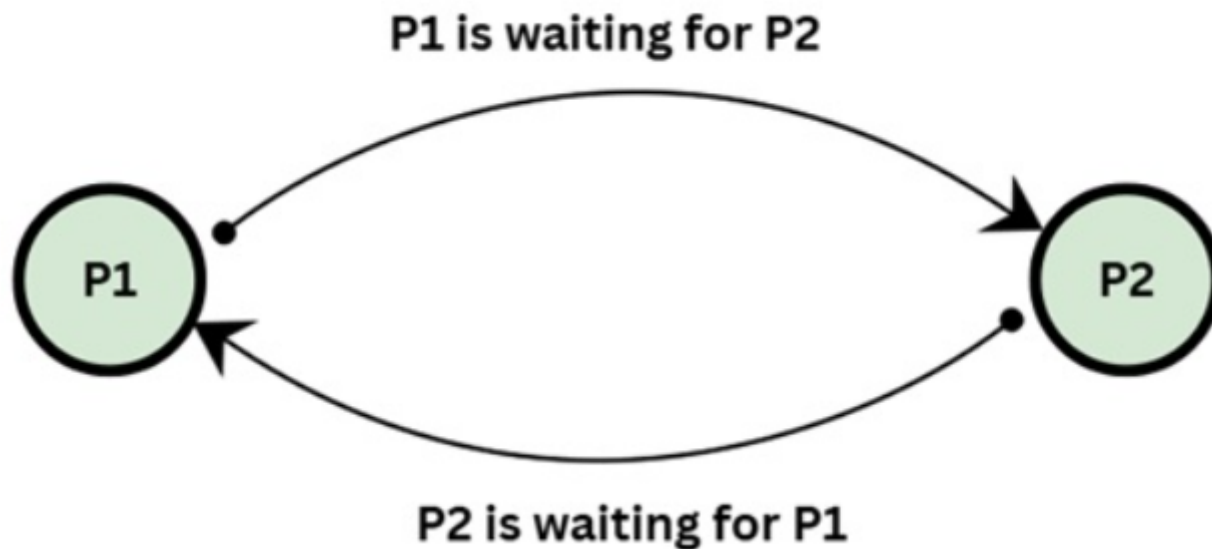
Resource Allocation Graph (RAG) Algorithm

- ▶ The Resource Allocation Graph (RAG) is a directed graph that represents the allocation of resources to processes and the requests made by processes for resources. In this graph, processes and resources are represented as nodes. Edges from a process to a resource represent a request for that resource, and edges from a resource to a process represent the allocation of that resource to the process.
- ▶ To detect deadlock using RAG, the operating system looks for cycles in the graph. If a cycle is found, it indicates that a deadlock has occurred among the processes involved in the cycle.



Wait-for Graph Algorithm

- ▶ The Wait-for Graph is a simplified version of the Resource Allocation Graph. In this graph, only processes are represented as nodes, and edges represent the waiting relationships between processes. An edge from process P1 to process P2 indicates that P1 is waiting for a resource held by P2.
- ▶ To detect deadlock using the Wait-for Graph, the operating system looks for cycles in the graph. If a cycle is found, it indicates that a deadlock has occurred among the processes involved in the cycle.



Note: The RAG and Wait-for Graph algorithms can only be used on systems with single-instance resources. i.e, each resource type has only one instance. For systems with multiple instances of resources, the Banker's Algorithm is used.

Deadlock recovery

- ▶ Deadlock recovery is the process of resolving a deadlock after it has already occurred. Unlike deadlock prevention (which avoids deadlocks in advance) or deadlock avoidance (which carefully allocates resources to avoid unsafe states), recovery accepts that deadlocks may occur and focuses on fixing them.
- ▶ **Deadlock Recovery Techniques**
- ▶ There are several ways an operating system can recover from a deadlock:
 1. **Process Termination**
 - ▶ The OS can break the deadlock by terminating one or more processes.
 - ▶ **Kill All Deadlocked Processes**
 - ▶ Simple but drastic: all processes involved in the deadlock are terminated.
 - ▶ Wasteful since a lot of work may be lost.
 - ▶ **Kill One Process at a Time (Victim Selection)**
 - ▶ The OS terminates one process at a time until the cycle is broken.
 - ▶ Factors considered for selecting the victim:
 - ▶ **Process priority**
 - ▶ How long the process has been running
 - ▶ How many resources it is holding
 - ▶ How much more work it needs to finish
 - ▶ Whether it's interactive or batch (interactive processes are less likely to be killed)

► 2. Resource Preemption

- Instead of killing processes, the OS may take away resources from one process and give them to another to break the cycle.
- **Challenges in resource preemption:**
- **Which resources to take?** Choosing the right resource is tricky.
- **Victim selection:** Pick a process whose rollback cost is lowest.
- **Starvation risk:** A process might repeatedly lose its resources (to avoid this, the OS tracks how many times a process has been chosen as a victim).